

Trabalho 3 >>>>>>

Projeto e Implementação do Jogo Pac-Man

Matheus Borges Nery De Mingo – 124675

Daniel Martins Teixeira – 124673

Rafael Vagner Pinto da Fonseca Souza – 124680

Manual de utilização do jogo



Para a movimentação do personagem principal, utilize as teclas W, para movimentar para cima, A, para movimentar para a esquerda, D, para movimentar para a direita, S, para movimentar para baixo

movimentação dos fantasmas

Os fantasmas possuem dois tipos de inteligência artificial, definidos pela variável **tipo** da struct **Fantasma**:

```
//Estrutura que representa um fantasma no jogo
struct Fantasma{
    float x, y;           //Posição atual do fantasma no mapa
    Direcao direcao;      //Direção atual de movimento
    sf::Sprite sprite;    //Imagem do fantasma que será desenhada na tela
    int tipo;             // 0 = Aleatorio, 1 = Perseguidor
    float velocidadeBase; //Velocidade de movimento do fantasma (Cada fantasma tem sua velocidade)
};
```

Cada inimigo possui posição, direção atual, sprite e velocidade própria, permitindo que cada inimigo tenha uma movimentação diferente

movimentação dos fantasmas

Antes de escolher uma direção, o jogo verifica quais movimentos são permitidos, evitando que os fantasmas atravessem paredes.

A função `fantasmaPodeMover()` analisa se a próxima posição é válida:

```
//Implementando restrições aos fantasmas
bool fantasmaPodeMover(int l, int c, Direcao d, int mapa[24][40]){
    if (d == Cima)
        l--;
    else if (d == Baixo)
        l++;
    else if (d == Esquerda)
        c--;
    else if (d == Direita)
        c++;

    // Impede que os fantasmas entrem no túnel
    if (l < 0 || l >= 24 || c < 0 || c >= 40)
        return false;

    // Proíbe que fantasmas entrem de volta na caixa (Parede Invisível)
    if (l == 11 && (c == 19 || c == 20) && d == Baixo)
        return false;

    return mapa[l][c] != 1; // Retorna true se não for parede
}
```

movimentação dos fantasmas

Depois, a função **pegarDirecoesValidas()** cria uma lista com todos os caminhos disponíveis:

```
// Retorna todas as direções válidas que o fantasma pode seguir a partir da posição atual
int pegarDirecoesValidas(int l, int c, Direcao atual, int mapa[24][40], Direcao opcoes[4]) {
    int qtd = 0;

    // Arrays auxiliares com as 4 direções e seus respectivos opostos
    Direcao dirs[4] = { Cima, Baixo, Esquerda, Direita };
    Direcao opostos[4] = { Baixo, Cima, Direita, Esquerda };

    for (int i = 0; i < 4; i++) {
        // Se o fantasma está parado (Nenhuma), ele pode ir para qualquer lado livre.
        // Se já está andando, ele só não pode ir para a direção oposta (voltar para trás).
        bool direcaoValida = (atual == Nenhuma) || (atual != opostos[i]);

        if (direcaoValida && fantasmaPodeMover(l, c, dirs[i], mapa)) {
            opcoes[qtd++] = dirs[i];
        }
    }

    // Retorna quantas direções válidas foram encontradas
    return qtd;
}
```

movimentação dos fantasmas

Os inimigos definidos como `int tipo = 0` escolhem uma direção aleatória quando encontram uma parede ou quando ainda não possuem direção definida

```
//IA dos inimigos aleatórios (Tipo 0)
if(fantasma[i].tipo == 0){
    //Se bateu na parede ou não tem direção definida, escolhe aleatoriamente
    if(naParede || fantasma[i].direcao == Nenhuma){
        if(qtdOpcoes > 0){
            fantasma[i].direcao = opcoesDeCaminho[numeroAleatorio(qtdOpcoes)];
        }
    }
}
```

A função `numeroAleatorio()` sorteia uma das direções disponíveis

Resumidamente, o inimigo anda pelo mapa, assim que encontra uma parede ou cruzamento verifica os caminhos livres, escolhe uma direção aleatória e continua andando até precisar escolher de novo

movimentação dos fantasmas

Os inimigos definidos como **int tipo = 1** analisam todas as direções possíveis e escolhem aquela que deixa eles o mais próximo do jogador.

```
//IA dos inimigos perseguidores (Tipo 1)
else if(fantasmas[i].tipo == 1){
    //Recalcula direção em casos necessários, que seriam colisão na parede, inatividade ou múltiplas opções(encruzilhadas)
    if (naParede || fantasmas[i].direcao == Nenhuma || qtdOpcoes > 1) {
        float menorDistancia = 999999.f;
        Direcao melhorDirecao = fantasmas[i].direcao;
        // Se não houver opções, fica parado
        if (qtdOpcoes == 0)
            melhorDirecao = Nenhuma;
```

```
//Testa todas as direções possíveis e escolhe a mais próxima do jogador
for(int k = 0; k < qtdOpcoes; k++){
    Direcao testarDir = opcoesDeCaminho[k];
    int nextL = lF, nextC = cF;
    //Simula a posição futura (1 bloco à frente) para avaliar cada direção possível
    if (testarDir == Cima)
        nextL--;
    else if (testarDir == Baixo)
        nextL++;
    else if (testarDir == Esquerda)
        nextC--;
    else if (testarDir == Direita)
        nextC++;
```

movimentação dos fantasmas

Os inimigos definidos como `int tipo = 1` analisam todas as direções possíveis e escolhem aquela que deixa eles o mais próximo do jogador.

```
//Calcula distância até o jogador
float dist = modulo(nextC * 40.f - posx) + modulo(nextL * 40.f - posy);
//Guarda a direção que deixa o fantasma mais próximo do jogador
if (dist < menorDistancia) {
    menorDistancia = dist;
    melhorDirecao = testarDir;
}
}
fantasmas[i].direcao = melhorDirecao; //Aplica a melhor direção encontrada, ou seja, a menor
```

Resumidamente, o inimigo verifica os caminhos possíveis, testa qual o caminho mais próximo do jogador e escolhe a direção com a menor distância

características adicionais

Adição de poderes (pílulas) ao personagem



Floco de neve: congela os inimigos por 4 segundos.

O valor **3** representa o floco de neve na matriz do mapa, quando o jogador pega o floco de neve ele desaparece, a pontuação aumenta em 80 pontos, a variável booleana **fantasmasCongelados** recebe **true** e o relógio **sf::Clock relógioCongelamento** começa a contar o tempo do efeito.

características adicionais

```
//Variáveis pro poder de congelamento  
bool fantasmasCongelados = false;  
sf::Clock relógioCongelamento;
```

```
//Sistema de usar o poder floco de neve (Número 3)  
else if (mapa[mtualLinha][atualColuna] == 3){  
    mapa[mtualLinha][atualColuna] = 0;  
    score += 80;  
    itensRestantes--;  
    textoPlacar.setString("SCORE: " + to_string(score));  
  
    fantasmasCongelados = true;  
    relógioCongelamento.restart();  
}
```

características adicionais

Antes de movimentar os inimigos, o código verifica se eles estão congelados

```
if (!fantasmasCongelados){  
    //Laço que cuida de um fantasma por vez  
    for (int i = 0; i < 4; i++) {
```

Enquanto `fantasmasCongelados` for verdadeiro, o código não executa a movimentação dos inimigos. Assim, os inimigos ficam completamente parados.

características adicionais

O efeito dura aproximadamente 4 segundos

```
//Lógica pra descongelar os inimigos depois de 4 segundos  
if(fantasmaCongelados && relógioCongelamento.getElapsedTime().asSeconds() >= 4.0f)  
    fantasmaCongelados = false;
```

Após o tempo acabar, o congelamento é desativado e os fantasmas voltam a se movimentar normalmente

características adicionais

Adição de poderes (pílulas) ao personagem



Relógio: deixa os inimigos em câmera lenta por aproximadamente 5 segundos

O valor **4** representa o relógio na matriz do mapa quando o jogador pega o relógio ele desaparece, a pontuação aumenta em 80 pontos, a variável **fantasmasLentos** recebe **true** e o contador do efeito é iniciado substituindo a velocidade dos inimigos

características adicionais

Adição de poderes (pílulas) ao personagem

```
//Variáveis pro poder de parar o tempo (relógio)
bool fantasmasLentos = false;
sf::Clock relógioLentidao;
```

```
//Usar o poder do relógio (Número 4)
else if (mapa[mtualLinha][atualColuna] == 4){
    mapa[mtualLinha][atualColuna] = 0;
    score += 80;
    itensRestantes--;
    textoPlacar.setString("SCORE: " + to_string(score));
    fantasmasLentos = true;
    relógioLentidao.restart();
}
```

```
//Define a velocidade dos fantasmas (reduzida quando o efeito de lentidão está ativo)
float velocidadeAtualFantasmas;
if(fantasmasLentos)
    velocidadeAtualFantasmas = 0.80f;
else
    velocidadeAtualFantasmas = fantasmas[i].velocidadeBase;
```

características adicionais

Espelhamento do personagem conforme a direção

Para que o personagem olhe para a direção em que está andando, usamos o espelhamento do sprite. Ao invés de carregar duas imagens diferentes (uma olhando para esquerda e outra para direita), o código utiliza a função `setScale()` da SFML para inverter a imagem horizontalmente. No jogo não colocamos o personagem olhando para cima nem para baixo

```
//Aplica o espelhamento vertical ao personagem principal, baseado na direção
if (direcaoAtual == Esquerda)
|   principal.setScale({-1.f, 1.f});           //Inverte X
else if (direcaoAtual == Direita)
|   principal.setScale({1.f, 1.f});           //Normal X
}
else
|   direcaoAtual = Nenhuma; // Interrompe o movimento ao colidir com uma parede
```

Identificação de cada parte do trabalho

Matheus Borges

- **Código das pílulas normais e dos poderes**
- **Movimentação dos fantasmas (assim como todos os outros membros)**
- **Construção e organização do PDF e arquivos adicionais**

Rafael Vagner

- **Criação do mapa**
- **Movimentação dos fantasmas (assim como todos os outros membros)**
- **Movimentação do personagem**

Daniel Martins

- **Criação das imagens e texturas utilizadas**
- **Movimentação dos fantasmas (assim como todos os outros membros)**
- **Código da pontuação, tela de vitória e tela de derrota**